

Computational Engineering

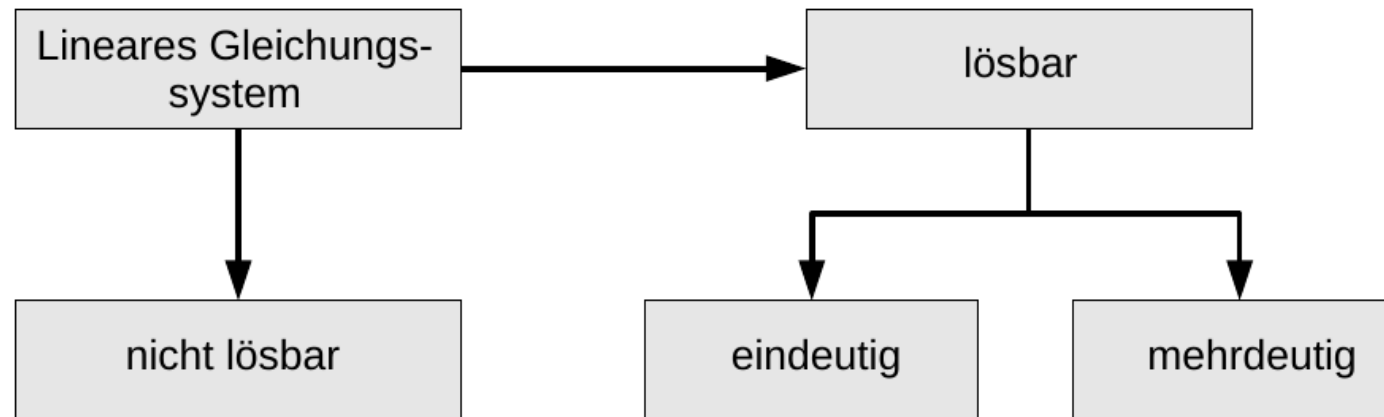
Lineare Gleichungssysteme

Lineare Gleichungssysteme gehören zu der grundlegenden Mathematik, die man in praktisch jeder quantitativen Wissenschaft antrifft und deshalb beherrschen muss.

Insbesondere numerische Algorithmen führen, ganz unabhängig von der ursprünglichen mathematischen Fragestellung, ganz häufig dazu, dass am Ende das Problem in die Lösung eines LGS umgewandelt wird. Beispiel hierzu sind insbesondere finite Differenz- und finite Element-Verfahren.

Kleine LGS, d.h. LGS die wenige Gleichungen und wenige Unbekannte enthalten, löst man ganz einfach per Hand mittels des Gaußalgorithmus. Moderne Anwendungen führen aber schnell auf riesige LGS von Millionen von Gleichungen für Millionen von Unbekannten. Es müssen also effiziente Verfahren gefunden werden diese LGS zu lösen.

Es gibt nur drei Möglichkeiten, wie die Lösungsmenge eines linearen Gleichungssystems aussehen kann



In der Praxis trifft man häufig auf quadratische lineare Gleichungssysteme, d.h. die Systeme haben gleich viele Gleichungen wie Unbekannte.

$$\begin{aligned}a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\&\vdots \\a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n\end{aligned}$$

Hierin sind $a_{ij} \in \mathbb{R}$ die **Koeffizienten**, b_i die **absoluten Glieder** und x_j die **Unbekannten** des linearen Gleichungssystems, jeweils für $i, j = 1, 2, \dots, n$.

Das lineare Gleichungssystem

$$a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2$$

$$\vdots$$

$$a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n$$

lässt sich auch in der **Matrix-Vektor Form** darstellen:

$$\mathbf{A} \cdot \vec{x} = \vec{b}$$

mit der Koeffizientenmatrix $\mathbf{A}$, dem Vektor der absoluten Glieder $\vec{b}$ und dem Lösungsvektor $\vec{x}$.

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}, \quad \vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

Bei der numerischen Lösung von linearen Gleichungssystem unterscheidet man zwei prinzipielle Verfahren.

- **Direkte Verfahren:** Führen in **endlich** vielen Schritten zu einer **exakten** (bis auf Rundungsfehler) Lösung des linearen Gleichungssystems. Diese Verfahren beruhen darauf, dass man ein lineares Gleichungssystem äquivalent in ein anderes, leichter zu lösendes, Gleichungssystem umformen kann. Man kennt das Verfahren als **Gaußalgorithmus**, welchen wir hier als bekannt voraussetzen.
- **Iterative Verfahren:** Erzeugen eine **Folge** von Lösungsvektoren $\vec{x}_k^T = (x_1, x_2, \dots, x_n)_k^T$, $k = 1, 2, \dots$ die gegen die Lösung des linearen Gleichungssystems konvergiert. Diese Verfahren erzeugen also eine **Näherungslösung**. Vorteil ist hier der idR deutlich geringere Rechenzeitverbrauch.

Gegeben sei das Gleichungssystem $\mathbf{A} \cdot \vec{x} = \vec{b}$. Der Gauß-Algorithmus besteht aus 2 wesentlichen Schritten:

- 1 Transformation des linearen Gleichungssystems in ein äquivalentes oberes Dreieckssystem.

$$\mathbf{A} \cdot \vec{x} = \vec{b} \quad \rightarrow \quad \mathbf{R} \cdot \vec{x} = \vec{\tilde{b}},$$

worin $\mathbf{R}$ eine obere Dreiecksmatrix ist. Dabei verwendet man nur die äquivalenten Umformungen

- Vertauschung zweier Zeilen.
 - Multiplikation einer Zeile mit einer reellen Zahl $\neq 0$.
 - Addition eines Vielfachen einer Zeile zu einer anderen.
- 2 Das Dreieckssystem kann dann durch sukzessives Einsetzen der einzelnen Komponenten der Lösung von unten nach oben vollständig gelöst werden.

Formulierung mittels erweiterter Koeffizientenmatrix

$$\left(\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & & & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{array} \right) \rightarrow \left(\begin{array}{cccc|c} r_{11} & r_{12} & \cdots & r_{1n} & \tilde{b}_1 \\ 0 & r_{22} & \cdots & r_{2n} & \tilde{b}_2 \\ \vdots & & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & r_{nn} & \tilde{b}_n \end{array} \right)$$

Dann sukzessives Auflösen

$$x_n = \frac{\tilde{b}_n}{r_{nn}}; \quad x_i = \frac{1}{r_{ii}} \left(\tilde{b}_i - \sum_{j=i+1}^n r_{ij} x_j \right), \quad i = n-1, n-2, \dots, 1.$$

Gauß-Algorithmus I: Diagonalmatrix

Input: $(\mathbf{A} = (a_{ji}), \vec{b} = (b_i))$

für $i = 1, \dots, n - 1$:

 wenn $a_{ii} \neq 0$:

 tue nichts;

 sonst

 wenn $a_{ji} = 0 \ \forall j = i + 1, \dots, n$:

$\mathbf{A}$ ist nicht regulär; stop

 sonst, wenn $a_{ji} \neq 0$ für ein $j = i + 1, \dots, n$:

 sei $j \geq i + 1$ der kleinste Index mit $a_{ji} \neq 0$;

 Vertausche Zeile(i) $\leftrightarrow$ Zeile(j);

 für $j = i + 1, \dots, n$:

$\text{Zeile}(j) \leftarrow \text{Zeile}(j) - \frac{a_{ji}}{a_{ii}} \cdot \text{Zeile}(i)$;

Output: $(\mathbf{R} = (r_{ij}), \vec{\tilde{b}} = (\tilde{b}_j))$

Gauß-Algorithmus II: Rücksubstitution

Input: $(\mathbf{R} = (r_{ij}), \vec{\tilde{b}} = (\tilde{b}_j))$

$$x_n = \frac{\tilde{b}_n}{r_{nn}};$$

für $i = n - 1, \dots, 1$:

$$x_i = \frac{1}{r_{ii}} \left(\tilde{b}_i - \sum_{j=i+1}^n r_{ij} x_j \right);$$

Output: $x_1, x_2, \dots, x_n$.

Der **Rechenaufwand** des Gauß-Algorithmus beträgt

$$\frac{2n^3 + 3n^2 + n}{6} \sim \mathcal{O}(n^3)$$

Punktoperationen. Es gibt weitere direkte Verfahren, die im wesentlichen Varianten des Gauß-Algorithmus sind und in der Regel nur für spezielle Arten von Koeffizientenmatrizen funktionieren, aber dafür weniger Rechenoperationen benötigen. Dies sind z.B.

- 1 Dreieckszerlegung
- 2 Cholesky-Verfahren
- 3 Verfahren für dünn besetzte Matrizen (sparse Matrizen)

Wir beschäftigen uns hier nun mit iterativen indirekten Verfahren zur Lösung eines LGS, die auf der Umwandlung in eine spezielle **Fixpunktform** basieren:

- Die Lösung des linearen Gleichungssystems $\mathbf{A} \cdot \vec{x} = \vec{b}$ ist gleichbedeutend mit der Lösung des linearen Nullstellenproblems $\mathbf{A} \cdot \vec{x} - \vec{b} = \vec{0}$.
- Idee ist es eine **Zerlegung** der Koeffizientenmatrix $\mathbf{A} = \mathbf{A}_1 + \mathbf{A}_2$ so zu definieren, dass man das Nullstellenproblem in eine **Fixpunktgleichung** umstellen kann.

$$\vec{x} = \mathbf{A}_1^{-1} \cdot (\vec{b} - \mathbf{A}_2 \cdot \vec{x}) = \mathbf{F}(\vec{x})$$

- Dafür muss $\mathbf{A}_1$ regulär sein und sollte eine “einfache” Struktur aufweisen.
- Damit definiert man dann eine **Fixpunktiteration**.
 - Festlegung eines **Startwertes** $\vec{x}_0$ für die Iteration.
 - Dann iterativ eine **Folge** von Werten durch

$$\vec{x}_{n+1} = F(\vec{x}_n), \quad n = 0, 1, 2, \dots$$

errechnen.

Gegeben sei ein lineares Gleichungssystem in Matrix-Form: $\mathbf{A} \cdot \vec{x} = \vec{b}$. Die Koeffizientenmatrix $\mathbf{A} = (a_{ij})$ sei zerlegt in der Form

$$\underbrace{\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}}_{\mathbf{A}} = \underbrace{\begin{pmatrix} 0 & \cdots & 0 & 0 \\ a_{21} & \ddots & 0 & 0 \\ \vdots & \ddots & \ddots & \vdots \\ a_{n1} & \cdots & a_{n,n-1} & 0 \end{pmatrix}}_{\mathbf{L}} \\
 + \underbrace{\begin{pmatrix} a_{11} & 0 & \cdots & 0 \\ 0 & a_{22} & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & a_{nn} \end{pmatrix}}_{\mathbf{D}} + \underbrace{\begin{pmatrix} 0 & a_{12} & \cdots & a_{1n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \ddots & a_{n-1,n} \\ 0 & 0 & \cdots & 0 \end{pmatrix}}_{\mathbf{R}}$$

Wir nennen diese Zerlegung die additive Diagonalzerlegung.

Sei $\mathbf{A} \cdot \vec{x} = \vec{b}$ ein gegebenes lineares Gleichungssystem und die Koeffizientenmatrix sei additiv diagonal zerlegt $\mathbf{A} = \mathbf{L} + \mathbf{D} + \mathbf{R}$.

Die Fixpunktiteration

- mit $A_1 = \mathbf{D}$ und $A_2 = \mathbf{L} + \mathbf{R}$:

$$\mathbf{D} \cdot \vec{x}_{n+1} = -(\mathbf{L} + \mathbf{R}) \cdot \vec{x}_n + \vec{b}$$

bzw.

$$\vec{x}_{n+1} = -\mathbf{D}^{-1} \cdot (\mathbf{L} + \mathbf{R}) \cdot \vec{x}_n + \mathbf{D}^{-1} \cdot \vec{b}$$

heißt **Gesamtschrittverfahren** oder auch **Jacobi-Verfahren**.

- mit $A_1 = \mathbf{D} + \mathbf{L}$ und $A_2 = \mathbf{R}$:

$$(\mathbf{D} + \mathbf{L}) \cdot \vec{x}_{n+1} = -\mathbf{R} \cdot \vec{x}_n + \vec{b}$$

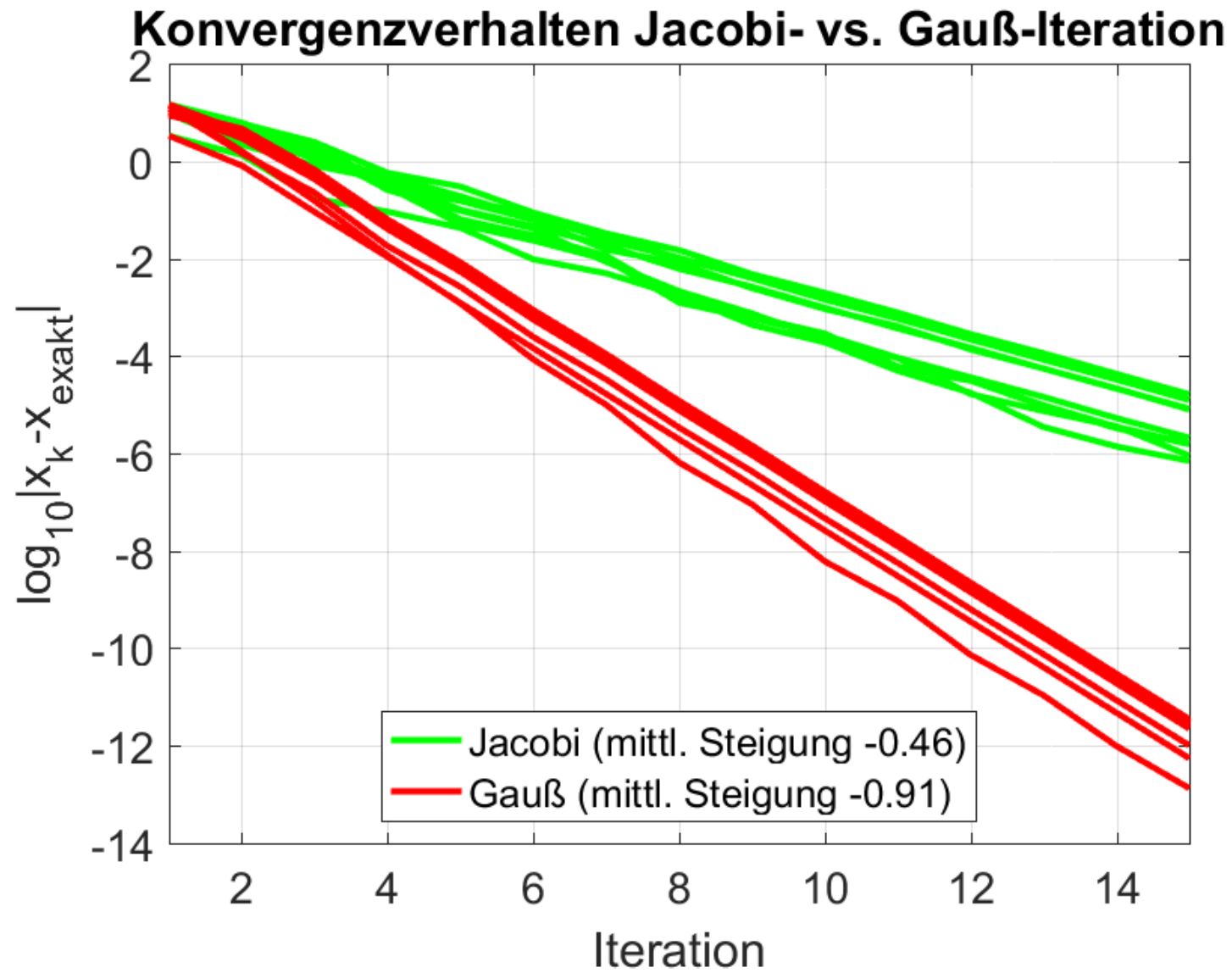
bzw.

$$\vec{x}_{n+1} = -(\mathbf{D} + \mathbf{L})^{-1} \cdot \mathbf{R} \cdot \vec{x}_n + (\mathbf{D} + \mathbf{L})^{-1} \cdot \vec{b}$$

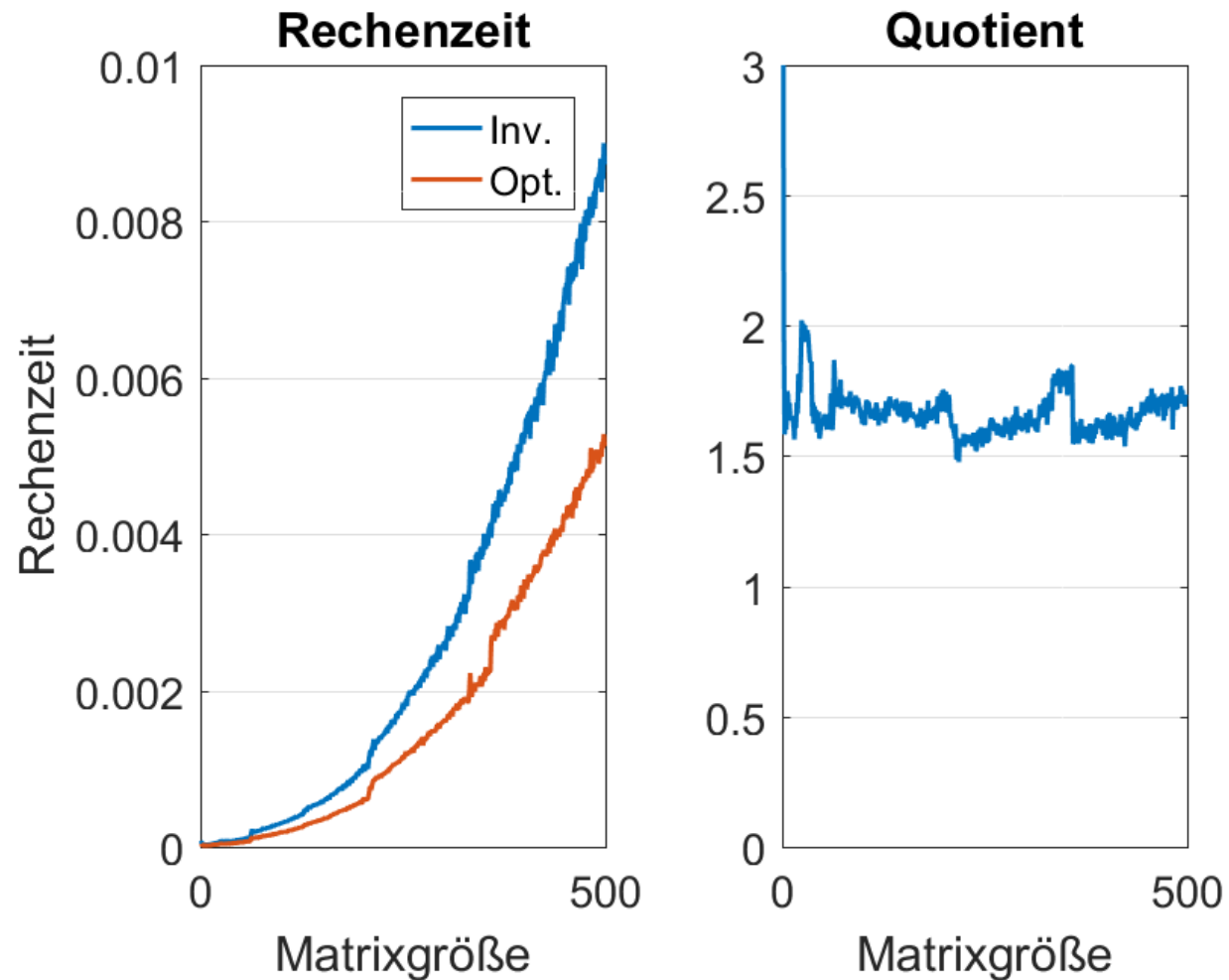
heißt **Einzelschrittverfahren** oder auch **Gauß-Seidel-Verfahren**.

Diese iterativen Methoden haben gegenüber dem direkten und prinzipiell exakten Gauß-Algorithmus zwei wesentliche Vorteile:

- Weniger Rechenoperationen. Da diese fixpunktbasierten Verfahren nur Matrix-Vektor Multiplikationen ausführen, beträgt der Rechenaufwand $\sim \mathcal{O}(n^2)$. also in etwa eine Größenordnung weniger, als bei den direkten Methoden. Da in der Praxis sehr große ($n \sim 10^6$ und mehr) Gleichungssysteme auftauchen, ist der Unterschied tatsächlich sehr relevant.
- In der Praxis tauchen Gleichungssysteme auf, in deren Koeffizientenmatrix hauptsächlich Nullen stehen, sogenannte dünn besetzte Systeme. Diese Nullen lassen sich speichertechnisch mit speziellen Algorithmen gut behandeln, so dass diese Systeme trotz ihrer nominellen Größe wenig Speicher verbrauchen. Beim Gauß Algorithmus werden aber bei solchen dünn besetzten Systemen in jedem Schritt Nullen zerstört, also durch Zahlen ersetzt, d.h. die Speichergröße eines Systems wächst im Laufe des Algorithmus i.A. enorm an (sogenanntes Fill-In-Problem). Bei den iterativen Methoden wird die Koeffizientenmatrix nie geändert und deshalb bleiben dünn besetzte Systeme auch dünn besetzt.



Vergleich der Rechenzeiten: LGS mittels Inverser vs. Matlab-optimiert (Backslash-Operator). Mittel über 100 runs.



Man kann ein lineares Gleichungssystem auch noch aus einer dritten Perspektive betrachten: Gesucht sind Zahlen $(x_1, x_2, \dots, x_n)^t = \vec{x}$, so dass

$$\mathbf{A} \cdot \vec{x} = \vec{b}$$

Wenn man also die Lösung $\vec{x}$ auf der linken Seite einsetzt, kommt auf der rechten Seite $\vec{b}$ heraus. Für einen beliebigen Vektor $\vec{x}'$, der nicht die Lösung ist, kommt natürlich eine andere rechte Seite raus:

$$\mathbf{A} \cdot \vec{x}' = \vec{b}'$$

Der Abstand der beiden rechten Seiten $\Delta = |\vec{b}' - \vec{b}|$ ist ein Maß dafür, wie nah man mit dem $\vec{x}'$ an der Lösung dran ist. Die Lösung des linearen Gleichungssystems kann aus dieser Perspektive als **Minimierungsproblem** betrachtet werden:

$$\min_{\vec{x}'} |\vec{b}' - \vec{b}| = \min_{\vec{x}'} |\mathbf{A} \cdot \vec{x}' - \vec{b}|$$

Als Beispiel betrachten wir noch einmal die **Wärmeleitungsgleichung**

$$T_i^{j+1} = \beta Q_i^j + \alpha \cdot (T_{i+1}^j + T_{i-1}^j) + (1 - 2\alpha)T_i^j$$

mit

$$\alpha = \frac{K}{\rho c} \frac{\Delta t}{(\Delta x)^2}, \quad \beta = \frac{\Delta t}{\rho c}, \quad T_0^j = T_n^j = 0$$

Diese Gleichung kann ebenfalls in Matrix-Vektor-Form geschrieben werden, wobei die **Ortsindizes** zu einem Vektor zusammen gefasst werden:

$$\vec{T}^{j+1} = \mathbf{A} \cdot \vec{T}^j + \beta \vec{Q}^j$$

mit der $(n-1) \times (n-1)$ Matrix

$$\mathbf{A} = \begin{pmatrix} (1-2\alpha) & \alpha & 0 & 0 & \cdots & 0 \\ \alpha & (1-2\alpha) & \alpha & 0 & \cdots & 0 \\ 0 & \alpha & (1-2\alpha) & \alpha & \cdots & 0 \\ \vdots & & & \ddots & & \vdots \\ 0 & \cdots & & 0 & \alpha & (1-2\alpha) \end{pmatrix}$$

Wenn die Temperaturverteilung im Laufe der zeitlichen Entwicklung gegen einen **Gleichgewichtszustand** strebt, dann würde für diesen Zustand ab einem sehr großen Zeitindex $j = J$ gelten

$$\vec{T}^{J+1} \approx \vec{T}^J$$

Der Gleichgewichtszustand $\vec{T}^J$ ist dann Lösung des linearen Gleichungssystems

$$(\mathbf{E} - \mathbf{A}) \cdot \vec{T}^J = \beta \vec{Q}^J$$

Auch das wollen wir in der Übung als Beispiel untersuchen.